UNIVERSIDADE FEDERAL DA BAHIA — UFBA

INSTITUTO DE COMPUTAÇÃO — DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

RELATÓRIO TÉCNICO DE AUDITORIA FORENSE E CRIPTOGRAFIA APLICADA

Proteção de Contratos Digitais, Cifras AEAD (AES-256-GCM), PKI X.509 e Assinaturas RSA

Disciplina: Segurança e Auditoria de Sistemas de Informação (MATC99)
Empresa Auditada (Cenário Prático): Miguelito Segurança e Tecnologia Ltda.
Autor: Wellington Miguel de Jesus Silva | **Data:** Novembro de 2026

---

1. Introdução e Contextualização do Incidente

A empresa **Miguelito Segurança e Tecnologia Ltda.** (CNPJ 14.382.910/0001-55), sediada em Salvador/BA, formaliza contratos eletrônicos com prestadores de serviços de infraestrutura crítica em nuvem. Durante a rotina de auditoria interna sobre a tramitação do **Contrato Comercial nº 2026/00849-MST**, firmado com a fornecedora *Parceiro Infraestrutura e Cloud Computing Ltda.* (parceiro.miguelito.com.br) no valor global de R$ 1.450.000,00, foram levantadas vulnerabilidades operacionais na transmissão e custódia do documento.

A auditoria delimitou três vetores de risco que demandavam tratamento criptográfico determinístico:

- **Adulteração em Trânsito (Bit-Flipping):** A possibilidade de interceptação de arquivos cifrados com manipulação de valores em modos de cifra maleáveis.
- **Fraude Pós-Assinatura:** A alteração intencional de cláusulas contratuais (ex.: parcela mensal de R$ 120.833,33 para R$ 920.833,33) após a aposição de assinatura legítima.
- **Falsificação de Identidade via Rogue CA:** A submissão de documentos assinados com chaves privadas associadas a certificados forjados emitidos por autoridades não confiáveis.

Para solucionar esses problemas, implementou-se uma arquitetura de proteção em duas frentes: confidencialidade e integridade via cifra simétrica autenticada **AES-256-GCM** (Parte I) e não-repúdio e garantia de identidade por meio de **PKI X.509** e assinaturas **RSA-SHA256** (Parte II).

2. Fundamentação Teórica

2.1. Criptografia Simétrica vs. Assimétrica

A criptografia simétrica emprega a mesma chave secreta para cifrar e decifrar. Apresenta alta taxa de transferência e suporte de hardware (AES-NI), sendo ideal para grandes volumes de dados. Contudo, não oferece não-repúdio entre as partes que conhecem a chave. Já a criptografia assimétrica baseia-se em pares de chaves matematicamente vinculadas (privada e pública). Embora seja computacionalmente mais exigente, resolve a distribuição de chaves e provê não-repúdio pericial.

2.2. Funcionamento do Modo AES-256-GCM (AEAD)

O AES-GCM (NIST SP 800-38D) é um esquema de cifragem autenticada que combina o modo contador (GCTR) com autenticação polinomial universal (GHASH) em GF(2^{12}■). O texto cifrado é gerado por $C_i = P_i \oplus E_K(inc$■■$(J$■$))$. Para garantir a integridade, deriva-se a subchave de autenticação $H = E_K(0^{12}$■$)$. Os dados associados (AAD) e os blocos de texto cifrado são avaliados segundo o polinômio irredutível $P(x) = x^{12}$■ + x■ + x^2 + x + 1. A Authentication Tag resultante de 128 bits é dada por:

$$T = GHASH_H(AAD, Ciphertext) \oplus E_K(J■)$$

Se um único bit de C_i for alterado (C'_i = C_i ⊕ Δ com Δ ≠ 0), a diferença na tag calculada será ΔT = Δ · H^(n - i + 2) mod P(x). Como GF(2¹²■) é um corpo finito isento de divisores de zero, ΔT ≠ 0 para qualquer Δ ≠ 0 e H ≠ 0, impedindo alterações silenciosas e disparando imediatamente a exceção *InvalidTag*.

2.3. Assinaturas Digitais (RSA-SHA256) e Infraestrutura de Chaves Públicas (RFC 5280)

A assinatura digital opera no paradigma Hash-then-Sign (PKCS#1 v1.5 / RFC 8017). Calcula-se o resumo h = SHA-256(M), aplica-se a codificação com preenchimento ASN.1 (EM) e eleva-se à chave privada: S = EM^d mod N. A verificação calcula EM' = S^e mod N e confere o hash. A PKI X.509 garante a amarração segura entre a identidade do titular e a chave pública através de uma cadeia de confiança hierárquica assinada pela Root CA corporativa.

3. Metodologia e Ambiente Experimental

Os testes foram desenvolvidos e executados em subsistema Linux de auditoria, preservando logs com carimbo de tempo em milissegundos e integridade forense SHA-256.

Componente	Versão / Especificação	Aplicação nos Experimentos
Sistema Operacional	Ubuntu 24.04 LTS (Kernel 6.6.87.2 WSL2)	Ambiente controlado de execução POSIX.
Interpretador	Python 3.12.3	Execução do módulo parte1_aes_gcm.py.
Biblioteca Cripto	cryptography 41.0.7 / 50.0.2	Cifragem AESGCM e tratamento de InvalidTag.
CLI OpenSSL	OpenSSL 3.0.13 (30 Jan 2024)	Geração de chaves, certificados X.509 e assinaturas.
Contrato Base	contrato_miguelito.txt (2.841 bytes)	Documento comercial nº 2026/00849-MST auditado.

4. Experimentos da Parte I: Proteção e Integridade com AES-256-GCM

O script parte1_aes_gcm.py carrega o contrato original (SHA-256: 9147c039...), gera chave simétrica de 256 bits, Nonce de 12 bytes (96 bits) e dados associados (AAD: MATC99-UFBA|MIGUELITO-SEGURANCA-E-TECNOLOGIA|CONTRATO-2026-00849). O arquivo cifrado contrato_miguelito.enc foi salvo em disco com 2.869 bytes (12B Nonce + 2.841B Ciphertext + 16B Tag).

A decifragem legítima recuperou os 2.841 bytes sem divergências (original == recuperado: True). Em seguida, foram testados os cenários de falha planejados:

```
[INFO] Contrato carregado: contrato_miguelito.txt (2841 bytes)
[INFO] SHA-256 original: 9147c03987856361e26a46f794f9588fe0850329cc2eaa28d5f0fd6ffe0cfe67
[INFO] Arquivo cifrado salvo em: contrato_miguelito.enc (2869 bytes totais)
[INFO] Comparacao de bytes (original == recuperado): True (2841 bytes)
[INFO] --- [Teste de Falha 1: Adulteracao de 1 byte no Ciphertext (Bit-Flipping)] ---
[INFO] Byte modificado no offset 1420 (0x058C): 0x4B -> 0x4A
[WARN] Excecao capturada com sucesso: cryptography.exceptions.InvalidTag
[INFO] Motivo: A alteracao mudou o resultado do GHASH em GF(2^128). Tag recalculada divergiu da original.
[INFO] --- [Teste de Falha 2: Tentativa de Decifragem com Chave Incorreta] ---
[WARN] Excecao capturada com sucesso: cryptography.exceptions.InvalidTag
[INFO] Motivo: Chave K' espuria altera subchave H' e mascara E_K'(J0), invalidando a tag em tempo constante.
```

5. Experimentos da Parte II: Assinaturas Digitais e Cadeia de Certificação

A infraestrutura PKI construída pelo script parte2_pki_testes.sh implantou duas hierarquias: (1) Cadeia Confiável corporativa composta pela **Miguelito Root CA** (RSA 4096 bits) e certificado da entidade **parceiro.miguelito.com.br** (RSA 2048 bits); (2) Cadeia Não Confiável formada por uma **Rogue CA** que emitiu certificado falso sob o mesmo CN do parceiro.

5.1. Cenário A: Documento Íntegro e Cadeia Confiável (Veredito: ACEITAR)

O contrato foi assinado pela chave privada legítima do parceiro e validado em duas etapas com a chave pública e a Trust Store corporativa:

```
$ openssl dgst -sha256 -verify parceiro_pub.pem -signature cenario_a.sig contrato_miguelito.txt
Resultado: Verified OK
$ openssl verify -CAfile miguelito_root_ca.crt parceiro.crt
Resultado: parceiro.crt: OK
```

Justificativa de Aceitação: Integridade matemática atestada pelo SHA-256 e identidade do signatário ratificada pela Root CA interna.

5.2. Cenário B: Adulteração Pós-Assinatura no Contrato (Veredito: REJEITAR)

Simulou-se fraude contratual modificando o valor mensal de R$ 120.833,33 para R$ 920.833,33 (1 byte alterado: '1' para '9'). O hash original (9147c039...) divergiu do hash adulterado (817d7fae...).

```
$ openssl dgst -sha256 -verify parceiro_pub.pem -signature cenario_a.sig contrato_adulterado.txt
error:02000068:rsa routines:ossl_rsa_verify:bad signature
Verification failure (Exit Code: 1)
```

Justificativa de Rejeição: Quebra estrita do hash SHA-256 pelo efeito avalanche. O bloco PKCS#1 decifrado não corresponde ao digest do texto alterado.

5.3. Cenário C: Certificado Forjado por Rogue CA (Veredito: REJEITAR)

O contrato íntegro foi assinado com a chave do certificado emitido pela Rogue CA. A verificação matemática isolada tem êxito, mas a validação de cadeia corporativa falha:

```
$ openssl dgst -sha256 -verify parceiro_forjado_pub.pem -signature cenario_c_forjado.sig contrato_miguelito.txt
Resultado: Verified OK  <-- Integridade matematica aparente
$ openssl verify -CAfile miguelito_root_ca.crt parceiro_forjado.crt
error 20 at 0 depth lookup: unable to get local issuer certificate
verification failed (Exit Code: 2)
```

Diferenciação Crítica: Demonstra-se que integridade matemática isolada não comprova legitimidade. Sem validação de cadeia X.509 na Trust Store, ataques de spoofing de certificado não são detectados.

6. Matriz de Decisão e Recomendações Corporativas

6.1. Tabela Comparativa de Auditoria

Cenário / Teste	Integridade Matemática	Confiabilidade da Chave / CA	Resultado OpenSSL / Python	Decisão
Parte I: AEAD Normal	Válida (Tag GCM confere)	Válida (Chave simétrica legítima)	2.841 bytes íntegros	**ACEITAR**
Parte I: Bit-Flipping	Inválida (GHASH diverge)	Válida (Chave legítima)	InvalidTag levantada	**REJEITAR**
Parte I: Chave Espúria	Não verificável (H' != H)	Inválida (Chave não autorizada)	InvalidTag levantada	**REJEITAR**
Parte II: Cenário A	Válida (Verified OK)	Válida (Miguelito Root CA: OK)	Exit Code 0 em ambos	**ACEITAR**
Parte II: Cenário B	Inválida (Verification failure)	Válida (Certificado legítimo)	Exit Code 1 (bad signature)	**REJEITAR**
Parte II: Cenário C	Válida (Verified OK)	Inválida (Fora da Trust Store)	Exit Code 2 (error 20)	**REJEITAR**

6.2. Recomendações Práticas para a Miguelito Segurança e Tecnologia

1. **Validação em Duas Etapas Obrigatória:** Condicionar a validação de contratos à checagem prévia da cadeia X.509 (-CAfile miguelito_root_ca.crt), recusando certificados emitidos por CAs não homologadas.
2. **Custódia da Root CA em HSM e Uso de Sub-CA:** Manter a chave privada RSA 4096-bit da Miguelito Root CA isolada (offline/HSM) e emitir certificados finais apenas por Autoridade Intermediária.
3. **Mecanismos de Revogação (CRL / OCSP):** Implementar listas CRL ou consulta OCSP no pipeline de recepção para barrar chaves privadas comprometidas antes do vencimento.
4. **Gestão de Nonces no AES-GCM:** Garantir Nonces aleatórios de 12 bytes gerados por CSPRNG sem repetição para uma mesma chave simétrica, evitando a recuperação da subchave H do GHASH.

7. Conclusão

A auditoria prática comprovou a complementaridade entre cifra simétrica autenticada e infraestrutura de chaves públicas. O AES-256-GCM assegurou a blindagem de trânsito contra interceptações e neutralizou ataques de bit-flipping via GHASH. Por sua vez, a PKI corporativa ancorada na Miguelito Root CA eliminou a fragilidade de validações matemáticas cegas (Cenário C), fornecendo a base necessária para que contratos digitais possuam confidencialidade, integridade, autenticidade e validade jurídica incontestável.

8. Referências Bibliográficas

1. DWORKIN, M. J. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, Nov. 2007.

2. COOPER, D. et al. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. IETF RFC 5280, Maio 2008.

3. MORIARTY, K. et al. *PKCS #1: RSA Cryptography Specifications Version 2.2*. IETF RFC 8017, Nov. 2016.

4. STALLINGS, W. *Criptografia e Segurança de Redes: Princípios e Práticas*. 8ª ed. São Paulo: Pearson, 2023.

5. PYTHON CRYPTOGRAPHIC AUTHORITY (PyCA). *Cryptography Documentation — AESGCM*. https://cryptography.io